

Introduction Document

Quantitative Development – Fall 2026

Trading at Georgia Tech

Kevin Lou, Justin Lee

1 Introduction

1.1 Document Goals

The purpose of this document is to introduce you to our tech stack and development process through a minimal project. The project should take a few hours at most. Once your project is accepted, you'll be given access to Trading at Georgia Tech repositories.

1.2 Stack Overview

Our primary programming language of choice is **Rust**. The reasoning is simple:

1. It lends itself to being used to write high-performance systems.
2. Unlike C++ and other C flavors, it encourages memory and thread safety by default.
3. Though subjective, it's pretty similar to C++.

In addition to Rust, we use **Python** for research and scripting, such as quoter deployment utilities. For research, we analyze and visualize market data using marimo notebooks.

1.3 Definitions

- **Quoter**: An instance of a trading engine. A quoter may manage a few, related strategies. Quoters are generally composed of listeners, strategies, and order gateways.
- **Listener**: A connection handler that manages a market data feed from an exchange. Listeners can be asynchronous or synchronous.
- **Strategy**: The algorithm that decides what, when, and how to trade based on the information fed to it by the dispatcher and pricing engine. Generally, it can be thought of as a black box that takes market data and theoretical prices as inputs and outputs a set of trading decisions based on some rules.
- **Order Gateway**: API for sending orders to a specific exchange based on that exchange's entry protocol.

1.4 Required Readings

We require all new members to read selected sections of the Rust Book so you can understand existing code.

To be blunt, it'll be pretty obvious if you're not familiar with the content covered in the assigned sections. Don't be the person tripping up on basic language errors – we have more important problems to solve! These are core concepts used in all of our systems.

The Rust book can be found here: <https://rust-book.cs.brown.edu/>

1. Getting Started
3. Common Programming Concepts
4. Understanding Ownership
5. Using Structs to Structure Related Data
6. Enums and Pattern Matching
7. Packages, Crates, and Modules
9. Error Handling
10. Generic Types, Traits, and Lifetimes
11. Writing Automated Tests
15. Smart Pointers
16. Fearless Concurrency
18. Object-Oriented Programming Features of Rust
20. Advanced Features

2 Warm-up Project

The **due date** for this warm-up project is **8:00PM on September 7th, 2026**.

2.1 Description

You are working on a cryptocurrency trading desk. A researcher wants to visualize the interesting price levels for ETH-USDT perpetual futures.

Your tasks are:

1. Take a look at the following exchange: [Woo X](#).
2. Ingest level 2 (i.e., 5 best bid/ask price levels) ETH-USDT perpetual futures data via the exchange websocket. There may be multiple websocket endpoints for getting order book data, pick the choice that gets us market data with the least delay.
3. Define an `Info` trait exposing an `info()` method, and write a procedural derive macro so that annotating a type with `#[derive(Info)]` generates an implementation printing each of its fields' names and types.
4. Derive `Info` on the types backing your order book.
5. Print the state of the order book in columnar format after every market data update.

Rust has no runtime reflection, so a field's type is only recoverable at compile time. Your macro's entry point will have the following signature:

```
#[proc_macro_derive(Info)]
pub fn derive_info(input: TokenStream) -> TokenStream {
    // parse `input` into a syn::DeriveInput, then emit the impl
}
```

Two structural notes: a procedural macro must live in its own crate, declared with `proc-macro = true` in its `Cargo.toml`, and the `Info` trait must be defined outside that crate, since a `proc-macro` crate can export nothing but macros. Use `syn` to parse the `TokenStream` and `quote` to emit the implementation.

2.2 Requirements

- Written in Rust, using the `Tungstenite` crate to connect to the exchange.
- Has a clean public API and is reasonably efficient.
- Includes well-written documentation for all public methods and associated functions.

2.3 Submission

Publish the final version on GitHub and email the repository link to both of us by **8:00PM on September 7th, 2026**:

- `klou@gatech.edu`
- `justinlee@gatech.edu`